

FINAL SUBMISSION PDF TEMPLATE (TASK-3)

Production-Grade Retrieval-Augmented Generation System

AI Internship Task-3 Submission

Name: Ayesha Abbas

1. PROJECT OVERVIEW

This project implements a **Production-Grade Retrieval-Augmented Generation (RAG) system** designed for enterprise document question answering.

The system combines:

- Hybrid Retrieval (Dense + Sparse)
- Reciprocal Rank Fusion (RRF)
- Cross-Encoder Re-ranking
- Query Rewriting
- Prompt Guardrails (Security Layer)
- LLM-based response generation
- Evaluation framework (RAGAS-style fallback metrics)

2. SYSTEM ARCHITECTURE

End-to-End Pipeline Flow:

User Query

↓

FastAPI (/chat endpoint)

↓

Security Guardrails (Injection Detection)

↓

Query Rewriting Module

↓

Hybrid Retrieval System

├— Dense Retrieval (FAISS)

├— Sparse Retrieval (BM25)

↓

RRF Fusion (Ranking Merge)

↓

Cross-Encoder Reranker

↓

Top-K Context Selection

↓

Prompt Builder

↓

LLM (Gemini API / Fallback)

↓

Final Answer + Sources

3. SYSTEM COMPONENTS

3.1 Data Ingestion Pipeline

- PDF document loading
- Text chunking using RecursiveCharacterSplitter
- Embedding generation
- FAISS vector index creation
- BM25 sparse index creation

3.2 Hybrid Retrieval System

- Dense retrieval using FAISS embeddings
- Sparse retrieval using BM25 keyword search
- Fusion using Reciprocal Rank Fusion (RRF)
- Relevance filtering based on query overlap

3.3 Query Understanding

- Query rewriting for ambiguity resolution
- Keyword expansion for better retrieval
- Input sanitization for security

3.4 Re-ranking Layer

- Cross-Encoder model (ms-marco MiniLM)
- Score-based ranking of retrieved chunks
- Top-K selection for context optimization

3.5 Generation Layer

- LLM-based answer generation (Gemini API)
- Context-grounded prompt building
- Source-aware response generation

- Hallucination minimized via strict prompting

3.6 Security & Guardrails

- Prompt injection detection
- Unsafe query filtering
- Input validation and sanitization
- Fallback safe responses

3.7 Evaluation Framework

- Faithfulness score
- Answer relevancy score
- Context recall score
- Evaluation performed using sample benchmark dataset

4. LATENCY PROFILING

Component	Time
Retrieval (Dense + Sparse)	~1.5s
Reranking	~0.3s
LLM Generation	~2.0–3.0s
Total End-to-End	~3–5 seconds

5. SECURITY ANALYSIS

The system includes multiple safety layers:

- Prompt injection detection before retrieval
- Blocking unsafe or irrelevant queries
- Input sanitization pipeline

- Controlled LLM prompt construction

Observed Behavior:

- Unsafe queries are rejected
- Malicious prompts are filtered
- System maintains stable outputs under adversarial inputs

6. EVALUATION RESULTS

Evaluation was performed on a **representative benchmark dataset (5–10 enterprise queries)**.

Metrics:

- Faithfulness Score: ✓ Moderate–High
- Answer Relevancy: ✓ Good
- Context Recall: ✓ Acceptable

Observation:

The system performs well on structured enterprise queries such as:

- onboarding policies
- cybersecurity rules
- performance evaluation
- leave policies

7. DEMO COVERAGE

The system demo includes:

- ✓ Document ingestion pipeline
- ✓ Swagger UI /docs interaction
- ✓ Query answering via /chat API
- ✓ Retrieval + response demonstration
- ✓ Evaluation pipeline execution

8. CONCLUSION

This project demonstrates a **production-style RAG system** with:

- Hybrid retrieval architecture
- Multi-stage ranking optimization
- LLM-based generation
- Security guardrails
- Evaluation framework

The system is scalable, modular, and designed for enterprise knowledge applications.